

RippleLab Day 08 Progress Report

Day: 08 of 30

Date: 2026-08-14

Status: Complete

Branch: codex/day-08-causal-graph

Implementation commit: c4b11d5

Deployed commit: c4b11d5

Production version: 4

Quality gate: Full local suite passed

1. Objective

Turn the Day 7 repo-rate result into RippleLab's first interactive causal graph. Day 8 must let a user trace the policy, loan, deposit and household paths; inspect any material node or edge; and see its mechanism, formula reference, assumptions, source, lag and confidence without creating a second economic model in the browser.

2. Acceptance Criteria

- [x] Render the engine-returned repo-rate graph with React Flow.
- [x] Distinguish policy, financial-product and household node categories.
- [x] Show directional edges with clear selected-state feedback.
- [x] Include zoom, zoom-out, fit-view and overview controls.
- [x] Open a synchronized evidence panel for every material node and edge.
- [x] Show mechanism, formula, assumptions, source and confidence for selections.
- [x] Show modeled lag ranges for causal edges.
- [x] Support pointer, touch, Tab, Enter and Space interaction.
- [x] Keep economic relationships read-only and prevent accidental editing.
- [x] Keep the graph usable without horizontal overflow on small screens.
- [x] Preserve the deterministic calculation boundary from Day 7.
- [x] Update and deploy the 8-of-30 progress experience.

3. Delivered Experience

After a signed-in user runs the repo-rate scenario, the normal cash-flow cards appear first. RippleLab then loads the causal graph as a separate client bundle and visualizes the exact causal graph included in the FastAPI result.

```
SimulationResult.causalGraph
-> categorized React Flow nodes
-> directional engine-returned edges
-> selected node or edge
-> synchronized evidence inspector
```

The user can select a graph element by pointer, touch or keyboard. The evidence inspector updates immediately while the calculated result and sensitivity cards remain unchanged.

4. Model and Presentation Boundary

Owned by the economic engine	Owned by the frontend
Node IDs, labels, values and units	Fixed spatial positions for the repo-rate view
Edge source, target and direction	Category colors and selected-state styling
Mechanisms and lag ranges	Human-readable value formatting
Assumption and citation references	Formula-reference labels for existing primitives
Confidence score and rationale	Viewport controls and responsive layout

The graph does not calculate rates, EMI, deposit interest or annual impact. Formula text is explanatory metadata for already-versioned Python primitives. If the engine returns a different relationship, the graph follows the returned node and edge contract.

5. Repo-rate Topology

The seeded result contains five material nodes and four causal edges.

Source	Target	Category transition	Formula reference
RBI repo rate	Floating loan rate	Policy -> financial product	<code>rates.linear_pass_through.v1</code>
Floating loan rate	Monthly EMI	Financial product -> household	<code>loan.floating_rate_reset.v1</code>
RBI repo rate	Deposit renewal rate	Policy -> financial product	<code>rates.linear_pass_through.v1</code>
Deposit renewal rate	Deposit income	Financial product -> household	<code>deposit.simple_interest.v1</code>

The node values reproduce the Day 7 seeded scenario: a -100 basis-point policy move, a modeled 7.80% loan rate, INR 61,369.23 monthly EMI, a modeled 6.75% deposit rate and an annual deposit-income effect of -INR 4,000.00.

6. Evidence Inspector

Selecting a node resolves its personal impact mechanism or nearest causal mechanism. Selecting an edge displays the mechanism returned for that relationship. The panel contains:

- node category or edge direction;
- formula expression and version identifier;
- every referenced assumption, selected value and rationale;
- every referenced citation, publisher and locator;
- edge lag range where applicable;
- confidence level, score and rationale.

For example, keyboard-selecting the floating-loan-rate-to-EMI edge shows `loan.floating_rate_reset.v1`, the selected loan pass-through, the remaining loan term, the Reserve Bank of India monetary-policy reference, the 0-1 month modeled lag and the edge-specific confidence explanation.

7. Interaction and Accessibility

React Flow's focusable nodes and edges are enabled. A keyboard user can Tab through the causal elements and press Enter or Space to inspect one. Focused nodes automatically pan into view. The zoom, zoom-out, fit-view and minimap controls have product-specific accessible labels.

The graph is deliberately read-only:

- nodes are not draggable;
- handles are not connectable;
- edges are not reconnectable;
- the Delete key is disabled;
- selection changes the inspector but never changes the economic model.

The scenario submit button is hydration-safe. It remains disabled with a preparation label until its client event handler is active, preventing a fast user interaction from being lost while the page hydrates.

8. Responsive and Loading Behavior

The graph and evidence inspector use a two-column workspace on wide screens. Below 920 pixels, the inspector moves underneath the canvas. At phone widths the graph viewport becomes shorter, the category legend wraps and the minimap is hidden so it cannot cover important nodes.

The React Flow bundle is dynamically imported only after a simulation result exists. This keeps the input form's initial client bundle smaller and avoids delaying the first scenario submission. A short loading card appears while the graph bundle becomes ready.

Automated mobile coverage selects the Deposit income node, verifies the inspector update and asserts that document width never exceeds the viewport.

9. Visual System

Policy nodes use RippleLab's deep ink color, financial-product nodes use teal and household nodes use amber. Default relationships use teal arrows. The selected relationship changes to amber and focused nodes receive a visible amber focus ring.

The evidence inspector uses the same deep ink surface as the deterministic net-impact card. Assumption values and confidence scores use warm signal color, while sources use a high-contrast teal link treatment.

The Day 8 social-preview card mirrors this structure: RBI policy rate branches to loan and deposit paths, and a selected relationship points to the evidence inspector motif.

10. Test and Verification Evidence

Check	Result
Economic-engine API and calculation suite	39 passed
Python lint and formatting	Passed
Canonical contracts and golden fixtures	Passed
Profile ownership, RLS, money and consent assertions	Passed
Repository secret scan	Passed
ESLint, generated routes and TypeScript	Passed
Next.js production build	Passed
Complete desktop and mobile browser suite	23 passed; 5 intentional device skips
Node selection and formula inspection	Passed
Keyboard edge selection, lag and RBI source inspection	Passed
Mobile graph and horizontal-overflow check	Passed
Result-state accessibility check	No serious or critical violations
Sites Vnext production bundle	Passed
Sites production deployment	Version 4 succeeded

The Python suite reports one existing dependency deprecation warning from FastAPI's Starlette test client regarding `ht tpx`. It does not affect the 39 passing tests.

11. Production Deployment

The Day 8 progress build is live at ripplelab-progress.nayaksiddhartha397.chatgpt.site. Sites version 4 was built, saved and deployed from exact pushed commit `c4b11d5`. Access remains owner-only during development.

The public product and progress surfaces are deployable. The authenticated simulator and causal graph are complete and verified locally, but they are not yet operational on the hosted site because production Supabase credentials and a hosted FastAPI endpoint have not been configured. No placeholder credentials were committed.

12. Important Files

- `apps/web/src/components/scenarios/causal-graph-explorer.tsx` - React Flow graph, selection and evidence inspector.
- `apps/web/src/components/scenarios/repo-rate-simulator.tsx` - hydration-safe form and lazy graph loading.
- `apps/web/src/app/globals.css` - graph, focus, inspector and responsive presentation.
- `apps/web/tests/repo-rate.spec.ts` - desktop keyboard and mobile graph coverage.
- `docs/CAUSAL_GRAPH_FRAMEWORK_V1.md` - graph boundary and interaction reference.
- `apps/web/src/app/progress/page.tsx` - Day 8 progress milestone.
- `apps/web/public/og.png` - Day 8 social-preview card.
- `apps/web/package.json` and `pnpm-lock.yaml` - official React Flow dependency.

13. Decisions and Limitations

- Use the official `@xyf low/react` package for graph interaction and accessibility support.
- Keep graph topology in the engine contract and layout metadata in the frontend.
- Keep the graph read-only until RippleLab supports an explicit scenario-authoring mode.
- Lazy-load the graph only after a result exists.
- Keep formula strings explanatory; never evaluate them in TypeScript.
- Retain the engine's edge-level confidence instead of assigning visual confidence.
- Hide the minimap on small screens while preserving zoom and fit controls.
- Link the current RBI evidence record without implying that it validates user-selected pass-through ratios.
- Continue treating all results as educational simulations, not financial advice.

No code blocker remains for Day 8.

14. Day 9 Handoff

Day 9 should add the first inflation engine. It should map a user-selected inflation change to a personal expense basket, use explicit category weights and pass-through assumptions, calculate annual household expense impact in the deterministic Python engine, and return a causal graph compatible with the Day 8 visual framework.

Sign-off: Prepared by Codex | Evidence reviewed: Yes | Ready to continue: Yes